

✓ Spark Preparation

We check if we are in Google Colab. If this is the case, install all necessary packages.

To run spark in Colab, we need to first install all the dependencies in Colab environment i.e. Apache Spark 3.3.2 with hadoop 3.3, Java 8 and Findspark to locate the spark in the system. The tools installation can be carried out inside the Jupyter Notebook of the Colab. Learn more from [A Must-Read Guide on How to Work with PySpark on Google Colab for Data Scientists!](#)

```
try:
    import google.colab
    IN_COLAB = True
except:
    IN_COLAB = False
```

```
print(IN_COLAB)
```

```
False
```

```
if IN_COLAB:
    !apt-get install openjdk-8-jdk-headless -qq > /dev/null
    !wget -q https://d1cdn.apache.org/spark/spark-3.3.2/spark-3.3.2-bin-hadoop3.tgz
    !tar xf spark-3.3.2-bin-hadoop3.tgz
    !mv spark-3.3.2-bin-hadoop3 spark
    !pip install -q findspark
    import os
    os.environ["JAVA_HOME"] = "/usr/lib/jvm/java-8-openjdk-amd64"
    os.environ["SPARK_HOME"] = "/content/spark"
```

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import avg, max
```

```
import os
import sys

os.environ['PYSPARK_PYTHON'] = sys.executable
os.environ['PYSPARK_DRIVER_PYTHON'] = sys.executable

# import findspark
# findspark.init()
```

✓ Start a Local Cluster

```
spark = SparkSession.builder.master("local").appName("App1").getOrCreate()
```

✓ Spark Assignment

Based on the movie review dataset in 'netflix-rotten-tomatoes-metacritic-imdb.csv', answer the below questions.

Note: do not clean or remove missing data

```
path = "netflix-rotten-tomatoes-metacritic-imdb.csv"
df = spark.read.csv(path=path, header=True, inferSchema=True)
df.printSchema()
```

```
root
|-- Title: string (nullable = true)
|-- Genre: string (nullable = true)
|-- Tags: string (nullable = true)
|-- Languages: string (nullable = true)
|-- Series or Movie: string (nullable = true)
|-- Hidden Gem Score: double (nullable = true)
|-- Country Availability: string (nullable = true)
|-- Runtime: string (nullable = true)
|-- Director: string (nullable = true)
```

```
-- Writer: string (nullable = true)
-- Actors: string (nullable = true)
-- View Rating: string (nullable = true)
-- IMDb Score: string (nullable = true)
-- Rotten Tomatoes Score: string (nullable = true)
-- Metacritic Score: string (nullable = true)
-- Awards Received: double (nullable = true)
-- Awards Nominated For: double (nullable = true)
-- Boxoffice: string (nullable = true)
-- Release Date: string (nullable = true)
-- Netflix Release Date: string (nullable = true)
-- Production House: string (nullable = true)
-- Netflix Link: string (nullable = true)
-- IMDb Link: string (nullable = true)
-- Summary: string (nullable = true)
-- IMDb Votes: string (nullable = true)
-- Image: string (nullable = true)
-- Poster: string (nullable = true)
-- TMdb Trailer: string (nullable = true)
-- Trailer Site: string (nullable = true)
```

```
cols = [c.replace(" ", "_") for c in df.columns]
df = df.toDF(*cols)
df.columns
```

```
['Title',
 'Genre',
 'Tags',
 'Languages',
 'Series_or_Movie',
 'Hidden_Gem_Score',
 'Country_Availability',
 'Runtime',
 'Director',
 'Writer',
 'Actors',
 'View_Rating',
 'IMDb_Score',
 'Rotten_Tomatoes_Score',
 'Metacritic_Score',
 'Awards_Received',
 'Awards_Nominated_For',
 'Boxoffice',
 'Release_Date',
 'Netflix_Release_Date',
 'Production_House',
 'Netflix_Link',
 'IMDb_Link',
 'Summary',
 'IMDb_Votes',
 'Image',
 'Poster',
 'TMdb_Trailer',
 'Trailer_Site']
```

- What is the maximum and average of the overall hidden gem score?

```
result = df.select(max("Hidden_Gem_Score"), avg("Hidden_Gem_Score")).collect()
m, a = result[0][0], result[0][1]
print(f"Max hidden gemscore: {m}, Average hidden gem score: {round(a,2)}")
```

Max hidden gemscore: 9.8, Average hidden gem score: 5.94

- How many movies that are available in Korea?

```
result = df.filter(df["Languages"].contains("Korea")).count()
print(f"Total Movie: {result}")
```

Total Movie: 735

- Which director has the highest average hidden gem score?

```
df_3 = df.groupBy("Director").agg(avg("Hidden_Gem_Score").alias("avg_hscore"))
max_val = df_3.select(max("avg_hscore")).collect()
q3 = df_3.filter(df_3["avg_hscore"] == max_val[0][0]).select("Director").collect()
print(f"The director whose have highest average hidden gem score is: {q3[0][0]}")
```

The director whose have highest average hidden gem score is: Dorin Marcu

✓ How many genres are there in the dataset?

```
df.filter(df["Genre"].isNull()).count()
```

1710

```
df_clean = df.filter(df["Genre"].isNotNull())
tolist = df_clean.select("Genre").rdd.flatMap(lambda x: x[0].split(",")).map(lambda x: x.strip()).map(lambda x: (x, 1)).reduce
print(tolist.count())
```

28